Examensarbete

Design och säkerhetanalys av OTA-uppdateringssystem för inbygdda fordonsenheter

Datum: 2026-01-22
Namn: Adewunmi Popoola
Utbildning: Applikationsingenör mot industri och fordon

Sammanfattning

Over-the-Air-uppdateringar (OTA) har blivit en central funktion i moderna fordonssystem där mjukvara behöver uppdateras, säkras och underhållas under hela fordonets livscykel. Detta examensarbete behandlar design, implementation och säkerhetsanalys av ett OTA-system anpassat för resursbegränsade inbyggda plattformar, med ESP8266 som representativ hårdvarubas.

Arbetet genomfördes i en simulerad miljö (Wokwi) efter att fysiska hårdvarutester avbröts på grund av återkommande fel i ESP8266-modulerna. Systemet utvecklades som en trelagersarkitektur bestående av en uppdateringsserver (Flask), ett TLS-skyddat kommunikationslager och en klient på enheten baserad på ett tillståndsdiagram med integritetskontroller (CRC32, SHA-256) och digital signaturverifiering (ECDSA P-256).

Resultaten visar att kompletta OTA-cykler kan genomföras på under sju sekunder för en firmwarefil på 295 kB, och att samtliga säkerhetskrav uppfylldes i simulering. En strukturerad hotanalys genomfördes med STRIDE-metoden och kompletterades med en jämförelse mot relevanta standarder såsom UNECE WP.29, ISO/SAE 21434, NIST SP 800-193 och Uptane.

Slutsatsen är att ett säkert och funktionellt OTA-system kan implementeras även på enklare mikrokontrollerplattformar, men att produktionsanvändning kräver hårdvarufunktioner som ESP8266 saknar, såsom Secure Boot, Flash Encryption och hårdvarubaserad nyckellagring.

INNEHÅLLSFÖRTECKNING

1. Inledning 5
1.1 Bakgrund 5
1.2 Problemformulering 5
1.3 Syfte och mål 6
1.4 Avgränsningar 6
1.5 Koppling till LIA 7

2. Teori 8
2.1 Inbyggda system i fordonsindustrin 8
2.2 OTA-uppdateringar 8
2.3 ESP8266-plattformen 9
2.4 Säkerhet vid firmwareuppdateringar 10
2.5 Simuleringbaserad utveckling 11

3. Metod 13
3.1 Forskningsansats 13
3.2 Simuleringsmiljö 13
3.3 Kravspecifikation 14
3.4 Teststrategi 15
3.5 Analysramverk 17

4. Resultat 18
4.1 Systemdesign 18
4.2 Implementation 18
4.3 Testresultat 19
4.4 Prestanda 19
4.5 Säkerhetsvalidering 20

5. Diskussion 21
5.1 Utvädering av resultat 21
5.2 Fordonsmässig genomförbarhet 21
5.3 Simulering kontra fysisk hårdvara 22
5.4 Begränsningar 23
5.5 Förslag på framtida arbete 23

6. Slutsatser 25
6.1 Sammanfattning av arbetet 25

6.2 Måluppfyllelse 25
6.3 Bidrag 25
6.4 Övergripande slutsats 26
6.5 Rekommendationer 26

7. Refrenslista 28

8. Bilagor 29
Bilaga A — ESP8266 hårdvaruspecifikation 29
Bilaga B — OTA-klientens källkod 31
Bilaga C — Säkerhetstester 36
Bilaga D — Wokwi-konfiguration 39
Bilaga E — Ordlista 42

1:Inledning

1.1 Bakgrund

Moderna fordon är i allt högre grad mjukvarudrivna system där funktioner, säkerhet och prestanda i stor utsträckning bestäms av den inbyggda elektroniken. Ett nutida fordon kan innehålla mellan 30 och över 150 elektroniska styrenheter (ECU:er), vilka tillsammans hanterar allt från motorstyrning och bromssystem till infotainment, uppkoppling och avancerade förarstödsfunktioner. Denna ökande komplexitet har gjort det nödvändigt att kunna uppdatera mjukvara kontinuerligt under fordonets livscykel.

Over-the-Air-uppdateringar (OTA) har därför blivit en central teknik inom fordonsindustrin. Genom OTA kan tillverkare distribuera säkerhetsuppdateringar, funktionsförbättringar och felrättningar utan att fordonet behöver besöka en verkstad. Tekniken introducerades brett av Tesla 2012 och har sedan dess blivit ett krav i branschen, inte minst genom UNECE WP.29/UN R156 som från 2022 gör OTA-kapacitet och Software Update Management Systems (SUMS) obligatoriska för typgodkännande i många regioner.

Trots fördelarna innebär OTA-uppdateringar betydande tekniska utmaningar. Många inbyggda system i fordon är resursbegränsade med begränsat minne, låg processorkraft och strikta krav på driftsäkerhet. Samtidigt måste uppdateringsprocessen vara robust, säker och kunna hantera avbrott utan att riskera att en ECU blir obrukbar. Dessa krav gör OTA-design till ett komplext ingenjörsproblem, särskilt på enklare mikrokontrollerplattformar.

Detta examensarbete behandlar utvecklingen av ett OTA-system för ESP8266 – en resursbegränsad men vanligt förekommande mikrokontroller inom IoT – med fokus på säkerhet, tillförlitlighet och fordonsmässig relevans.

1.2 Problemformulering

Att implementera ett säkert och tillförlitligt OTA-system på en resursbegränsad plattform innebär flera utmaningar:

- Begränsad hårdvara: ESP8266 har begränsat RAM, begränsad flash och saknar hårdvarufunktioner som Secure Boot och Flash Encryption. Detta påverkar möjligheten att använda traditionella OTA-metoder såsom dubbla firmwarepartitioner och avancerade kryptografiska kontroller.

- Säkerhetskrav: OTA-kanalen är en potentiell angreppspunkt. Utan korrekt autentisering, integritetskontroll och skydd mot replay- och rollback-attacker kan en angripare injicera skadlig firmware eller manipulera uppdateringsflödet.

- Tillförlitlighet: En avbruten eller felaktig uppdatering får inte göra enheten obrukbar. Systemet måste kunna hantera avbrott, verifiera firmware och återgå till en fungerande version vid behov.

- Simuleringsbegränsningar: På grund av hårdvaruproblem under projektets inledande fas genomfördes implementationen i en simulerad miljö. Detta skapar en metodologisk utmaning i att säkerställa att resultaten är relevanta för verklig hårdvara.

Problemet som undersöks är därför:
Hur kan ett säkert, tillförlitligt och resurseffektivt OTA-system designas och implementeras på en plattform som ESP8266, och i vilken utsträckning kan ett sådant system anses vara tillämpbart i fordonsmiljöer?

1.3 Syfte och mål

Syftet med examensarbetet är att utveckla och analysera ett OTA-system som uppfyller centrala krav på säkerhet, funktionalitet och tillförlitlighet, med särskilt fokus på fordonsrelaterade användningsfall.

Arbetet har följande mål:

- Designa en OTA-arkitektur anpassad för ESP8266 med tydligt definierade komponenter, kommunikationsflöden och säkerhetsmekanismer.
- Implementera systemet i en simuleringsmiljö (Wokwi) för att demonstrera OTA-funktionalitet utan beroende av fysisk hårdvara.
- Genomföra en säkerhetsanalys baserad på STRIDE, CVSS och relevanta standarder såsom UNECE WP.29 och ISO/SAE 21434.
- Utvärdera systemetsfordonsmässiga relevans, inklusive begränsningar och krav för att kunna användas i verkliga ECU-miljöer.
- Dokumentera systemet enligt Hermods YH-standard för examensarbeten.

1.4 Avgränsningar

Arbetet avgränsas till följande:

- Endast en mikrokontrollerplattform (ESP8266) analyseras och implementeras.
- Implementation sker i simulerad miljö, inte på fysisk hårdvara.
- Fokus ligger på OTA-uppdatering av firmware, inte applikationsdata eller konfigurationsfiler.
- Arbetet inkluderar inte:
- multi-ECU-uppdateringar

- AUTOSAR-baserade system
- hårdvarubaserade säkerhetsmoduler (HSM)
- RF-tester eller verkliga Wi-Fi-miljöer
- fullständig implementation av Uptane

Dessa avgränsningar gör det möjligt att fokusera på kärnfrågan: hur ett säkert OTA-system kan byggas på en resursbegränsad plattform.

1.5 Koppling till LIA

Arbetet bygger på erfarenheter från LIA-perioden på Edgework, där jag arbetade som teknisk skribent och testare inom ett mjukvaruutvecklingsteam. Under LIA:n fick jag praktisk erfarenhet av:

- dokumentation av inbyggda system
- kvalitetssäkring och testning
- systemintegration
- hantering av sensordata och inbyggda plattformar

Dessa erfarenheter har varit direkt användbara i detta examensarbete, särskilt i utformningen av teststrategi, dokumentationsstruktur och förståelsen för OTA-flöden i distribuerade system.

2: Teori

2.1 Inbyggda system i fordonsindustrin

Moderna fordon är komplexa distribuerade datorsystem där ett stort antal elektroniska styrenheter (ECU:er) samverkar för att styra fordonets funktioner. En ECU består vanligtvis av en mikrokontroll, intern och extern minnesenhet, kommunikationsgränssnitt samt dedikerad programvara. Antalet ECU:er i ett fordon varierar mellan cirka 30 och över 150 beroende på modell och utrustningsnivå.

ECU:er delas ofta in efter funktionella domäner:

- Drivlina: motorstyrning, växellåda, bränsleinsprutning
- Chassi: ABS, ESC, styrning, bromssystem
- Kaross: belysning, låssystem, klimatkontroll
- Infotainment och telematik: navigering, uppkoppling, OTA-hantering
- ADAS: sensorfusion, kameror, radar, lidar

Kommunikationen mellan ECU:er sker via standardiserade fordonsnätverk. De vanligaste är:

- CAN (Controller Area Network): robust, deterministiskt och fortfarande dominerande
- LIN (Local Interconnect Network): enklare och billigare, används för icke-kritiska funktioner
- FlexRay: hög bandbredd och determinism för avancerade chassifunktioner
- Automotive Ethernet: växande standard för ADAS, infotainment och OTA-distribution

Traditionellt har ECU-uppdateringar utförts via UDS (Unified Diagnostic Services) genom OBD-porten i verkstadsmiljö. Detta kräver fysisk åtkomst, utbildad personal och är tidskrävande vid stora fordonsflottor. Övergången till OTA-uppdateringar är därför en naturlig utveckling i takt med att fordon blir mer mjukvarudrivna.

2.2 Over-The-Air (OTA) Uppdateringar

OTA-uppdateringar innebär att firmware, mjukvara eller konfigurationsdata distribueras trådlöst till en enhet utan fysisk åtkomst. Inom fordonsindustrin skiljer man ofta mellan:

- SOTA (Software-Over-the-Air): uppdatering av applikationsmjukvara
  - FOTA (Firmware-Over-the-Air): uppdatering av ECU-firmware

Ett komplett OTA-system består av tre huvuddelar:

- Backend: uppdateringsserver, firmwarelager, kampanjhantering
- Kommunikationslager: TLS-skyddad överföring via Wi-Fi eller mobilnät
- Enhetsklient: versionkontroll, nedladdning, verifiering, partitionering, rollback

Fördelarna med OTA är betydande:

- snabb distribution av säkerhetsuppdateringar
- minskade kostnader för återkallelser
- förbättrad kundupplevelse genom kontinuerliga funktionsförbättringar
- möjlighet till fjärrdiagnostik och fleet management

Utmaningarna är dock omfattande:

- varierande nätverkskvalitet
- krav på strömsäkerhet under uppdatering
- hårdvarubegränsningar i ECU:er
- säkerhetsrisker såsom replay-attacker, AitM-attacker och rollback-angrepp

Standarder som Uptane (RFC 9334), TLS 1.2/1.3, AUTOSAR UCM och UNECE WP.29 definierar krav och rekommendationer för säkra OTA-system.

2.3 ESP8266-plattformen

ESP8266 är en lågkostnads-SoC utvecklad av Espressif Systems. Den är vanligt förekommande i IoT-projekt och används i detta arbete som representativ plattform för resursbegränsade inbyggda system.

Centrala egenskaper:

- CPU: Tensilica L106, 32-bit, upp till 160 MHz
- RAM: 80 kB
- Extern flash: 1–4 MB
- Kommunikation: 2.4 GHz Wi-Fi (802.11 b/g/n)
- Gränssnitt: UART, SPI, I2C, GPIO, ADC

För OTA-uppdateringar erbjuder ESP8266-Arduino-kärnan ett A/B-partitionssystem på enheter med 4 MB flash. Firmware laddas ned till den inaktiva partitionen och aktiveras först efter verifiering. TLS-kommunikation hanteras av BearSSL, som är anpassad för små minnesmängder genom reducerade buffertstorlekar.

Begränsningar som påverkar säker OTA:

- ingen hårdvarubaserad Secure Boot
- ingen flash-kryptering
- begränsat RAM för kryptografiska operationer
- känslighet för strömavbrott under flashning

Trots detta är plattformen tillräckligt kapabel för att demonstrera ett komplett OTA-flöde.

2.4 Säkerhet vid firmwareuppdateringar

Firmwareuppdateringar är en kritisk attackvektor. Vanliga hot inkluderar:

- Replay-attacker: en gammal, sårbar firmware återanvänds
- AitM-attacker: firmware byts ut under överföring
- Rollback-attacker: angriparen tvingar enheten att starta en äldre version
- Serverkompromettering: angriparen manipulerar firmwarelagret

För att motverka dessa används flera säkerhetslager:

- Transportskydd: TLS 1.2/1.3 med ECDHE-AES-GCM
- Firmwareautentisering: ECDSA-signaturer över SHA-256-hash
- Integritetskontroll: CRC32 eller SHA-256
- Enhetsautentisering: X.509-certifikat eller PSK
- Versionspolicy: anti-rollback-skydd
- Manifestformat: IETF SUIT för strukturerade metadata

I fordonsmiljöer är dessa krav reglerade av standarder som:

- ISO/SAE 21434: cybersäkerhet för fordon
- UNECE WP.29: krav på SUMS och CSMS
- NIST SP 800-193: plattformsresiliens
- Uptane: säker OTA-arkitektur för fordon

2.5 Simuleringsbaserad utveckling

På grund av hårdvaruproblem under projektets inledande fas genomfördes implementationen i Wokwi, en simuleringsmiljö som kör verkliga firmwarebinärer. Detta möjliggör:

- reproducerbara tester
- säker testning av felmoder
- minnesprofilering utan risk för hårdvarubrickning
- snabbare utvecklingscykler

Begränsningar i simuleringen:

- ingen fysisk observation av flashskrivningar
- idealiserad Wi-Fi-miljö

- ingen modellering av RF-störningar
- ingen möjlighet att testa Secure Boot eller flash-kryptering

Trots dessa begränsningar är simuleringen tillräckligt realistisk för att validera OTA-logik, säkerhetsflöden och prestanda.

3: Methodology

3.1 Forskningsansats

Arbetet utgår från *Design Science Research* (DSR), en forskningsansats som fokuserar på att utveckla och utvärdera tekniska artefakter som löser identifierade problem. I detta examensarbete utgörs artefakten av ett komplett OTA-system bestående av:

- en backendserver för versionering och firmwaredistribution
- ett kommunikationslager baserat på HTTPS/TLS
- en OTA-klient på ESP8266 med verifieringslogik och tillståndsmaskin

DSR-processen följer tre centrala steg:

- Problemidentifiering: resursbegränsningar, säkerhetskrav och tillförlitlighetsbehov vid OTA-uppdateringar på enkla mikrokontrollerplattformar.
- Artefaktutveckling: design och implementation av ett OTA-system i simulerad miljö.
- Utvärdering: funktionella tester, prestandamätningar och säkerhetsanalys enligt STRIDE och CVSS.

Valet av DSR motiveras av att arbetet syftar till att *skapa* och *validera* en teknisk lösning snarare än att enbart analysera befintliga system.

3.2 Simuleringsmiljö

På grund av återkommande hårdvarufel i ESP8266-modulerna under projektets inledande fas genomfördes implementationen i Wokwi, en simuleringsmiljö som kör verkliga firmwarebinärer. Detta möjliggjorde:

- stabil utvecklingsmiljö utan risk för hårdvarubrickning
- reproducerbara tester
- möjlighet att simulera felmoder som annars vore riskabla
- snabbare iterationer

Utvecklingsmiljön bestod av:

- Visual Studio Code 1.87
- PlatformIO 6.1
- Wokwi VS Code-extension 2.1
- Python Flask-server för versionmanifest och firmwaredistribution
- OpenSSL för generering av testcertifikat

Simuleringen använder en virtuell Wi-Fi-brygga som gör det möjligt för ESP8266-firmware att kommunicera med en lokal server på utvecklingsdatorn.

3.3 Kravspecifikation

Kravspecifikationen baserades på tre källor:

- problemformuleringen
- fordonsrelaterade säkerhetsstandarder
- ESP8266-plattformens begränsningar

Krav delades in i tre kategorier:

Funktionella krav (FR)

Exempel:

- FR-01: Enheten ska kunna ansluta till Wi-Fi.
- FR-02: Enheten ska kunna hämta versionsinformation via HTTPS.
- FR-03: Firmware ska laddas ned i segment (chunked download).
- FR-04: Integritetskontroll ska utföras med CRC32.
- FR-05: Firmware ska verifieras med SHA-256 och ECDSA.
- FR-06: Systemet ska kunna utföra rollback vid fel.

Icke-funktionella krav (NFR)

Exempel:

- NFR-01: Uppdateringen får inte göra enheten obrukbar.
- NFR-02: Minnesanvändningen får inte överstiga 60 kB under kritiska faser.
- NFR-03: Nedladdning av firmware ska ske inom 120 sekunder.
- NFR-04: Systemet ska vara kompatibelt med ESP8266 Arduino Core 3.x.

Säkerhetskrav (SR)

Exempel:

- SR-01: All kommunikation ska ske över TLS 1.2.
- SR-02: Certifikatpinning ska användas.
- SR-03: Firmware ska signeras med ECDSA P-256.
- SR-04: Anti-rollback-skydd ska implementeras.
- SR-05: Uppdateringen ska avbrytas vid minsta verifieringsfel.

Kravspecifikationen användes som grund för både design och testning.

3.4 Teststrategi

Testningen delades in i tre huvudkategorier:

Funktionella tester

Dessa verifierade att alla funktionella krav uppfylldes. Testerna genomfördes genom att observera:

- seriell loggning
- tillståndsövergångar i FSM

- serverrespons
- firmwarebeteende efter omstart

Totalt definierades tolv funktionella testfall.

Prestandatester

Mätningar utfördes för:

- nedladdningstid
- verifieringstid (CRC, SHA-256, ECDSA)
- minnesanvändning under kritiska faser

Dessa mätningar gjordes med hjälp av:

- ESP.getFreeHeap()
- tidsstämplar i seriell logg

Säkerhetstester

Säkerhetsvalideringen bestod av:

- kodgranskning
- hotmodellering (STRIDE)
- sårbarhetsbedömning (CVSS v3.1)
- jämförelse mot standarder (UNECE WP.29, ISO/SAE 21434, Uptane)

Simulerade attacker inkluderade:

- manipulerade firmwarefiler
- felaktiga signaturer
- avbrutna nedladdningar

- versioner som bryter anti-rollback-regler

3.5 Analysramverk

Analysen baserades på tre etablerade ramverk:

STRIDE

Användes för att identifiera hot inom sex kategorier:

- Spoofing
- Tampering
- Repudiation
- Information Disclosure
- Denial of Service
- Elevation of Privilege

CVSS v3.1

Användes för att bedöma allvarlighetsgrad hos identifierade sårbarheter.
Varje sårbarhet fick en numerisk score och en klassificering (Low, Medium, High, Critical).

Standardjämförelse

Systemet jämfördes mot krav i:

- UNECE WP.29 (SUMS/CSMS)
- ISO/SAE 21434 (cybersäkerhet för fordon)
- NIST SP 800-193 (plattformresiliens)
- Uptane (OTA-säkerhet för fordon)

Analysen syftade till att bedöma hur väl systemet uppfyller fordonsmässiga krav och vilka gap som kvarstår.

4: Resultat

4.1 Systemdesign

Systemet som utvecklades består av en trelagers OTA-arkitektur anpassad för ESP8266 och simulerad i Wokwi. Arkitekturen omfattar:

- **Backendserver:** Python Flask-applikation som tillhandahåller versionsmanifest och firmwarefiler via HTTPS.
- **Kommunikationslager:** TLS 1.2-skyddad överföring med certifikatpinning för att förhindra AitM-attacker.
- **Enhetsklient:** OTA-klient implementerad som en tillståndsmaskin (FSM) med sju definierade tillstånd, ansvarig för versionkontroll, nedladdning, verifiering och aktivering av firmware.

Designen följer en *pull-modell*, där enheten själv initierar alla uppdateringsförfrågningar. Detta är lämpligt för resursbegränsade enheter med intermittent nätverksanslutning.

Systemet innehåller fyra centrala komponenter:

- **Nätverkshanterare:** Wi-Fi-anslutning, TLS-konfiguration och certifikatpinning.
- **OTA-klient:** styr logiken för versionkontroll, nedladdning och uppdateringsflöde.
- **Säkerhetsmodul:** CRC32-kontroll, SHA-256-hashning och ECDSA-signaturverifiering.
- **Partitionshanterare:** hanterar ESP8266:s A/B-partitioner och rollback-funktionalitet.

Designen säkerställer att enheten aldrig skriver över aktiv firmware innan verifiering är genomförd, vilket minimerar risken för att enheten blir obrukbar.

4.2 Implementation

Implementation genomfördes i Wokwi-simulatorn med ESP8266-Arduino-kärnan. Backendservern utvecklades i Python Flask och exponerade två REST-endpoints:

- **GET /version** – returnerar JSON-manifest med firmwareversion, hash och signatur.
- **GET /firmware/** – returnerar firmwarebinären i chunked format.

Den simulerade ESP8266-klienten implementerades i C++ med följande funktioner:

- **TLS-anslutning via BearSSL** med 4 kB buffertar för att passa i ESP8266:s begränsade RAM.
- **Chunked nedladdning** för att undvika minnesöverflöd.
- **SHA-256-hashning** av firmware i realtid under nedladdning.

- **ECDSA P-256-verifiering** av firmware innan aktivering.
- **Rollback-logik** som återställer tidigare firmware vid fel.

Simuleringen körde verkliga firmwarebinärer, vilket gjorde det möjligt att testa hela OTA-flödet utan fysisk hårdvara.

4.3 Testresultat

Totalt genomfördes tolv funktionella tester. Resultaten sammanfattas nedan:

- **11 tester godkända**
- **1 test delvis godkänt** (begränsning i simuleringen)

Exempel på testfall:

- **Wi-Fi-anslutning:** Enheten anslöt stabilt i samtliga testkörningar.
- **Versionskontroll:** JSON-manifest hämtades korrekt via HTTPS.
- **Nedladdning:** Firmware laddades ned i segment utan minnesproblem.
- **CRC-kontroll:** Felaktiga firmwaresegment upptäcktes och avbröt uppdateringen.
- **SHA-256-verifiering:** Hashvärden matchade serverns manifest.
- **ECDSA-verifiering:** Signaturer verifierades korrekt; manipulerade firmwarefiler avvisades.
- **Rollback:** Vid simulerat fel återgick enheten till tidigare firmware.

Det enda delvisa testet gällde observation av fysiska flashskrivningar, vilket inte stöds i Wokwi. Funktionaliteten verifierades istället genom loggar och partitionsstatus.

4.4 Prestanda

Prestandamätningar utfördes för en firmwarefil på 295 kB. Resultaten visar:

- **Total OTA-tid:** < 7 sekunder
- **Nedladdningstid:** ~4,2 sekunder
- **SHA-256-hashning:** ~1,1 sekunder
- **ECDSA-verifiering:** ~0,9 sekunder
- **Minsta fria heap under TLS-handshake:** 27 kB

Dessa resultat visar att även en resursbegränsad plattform som ESP8266 kan genomföra OTA-uppdateringar snabbt och stabilt, förutsatt att firmwarestorleken hålls relativt liten.

Minnesanvändningen höll sig under det definierade kravet på 60 kB under hela processen.

4.5 Säkerhetsvalidering

Säkerhetsvalideringen bestod av:

- **STRIDE-analys**
- **CVSS-bedömning av identifierade sårbarheter**
- **Jämförelse mot fordonsstandarder**

Åtta sårbarheter identifierades och bedömdes enligt CVSS v3.1. Samtliga kunde hanteras genom implementerade kontroller eller genom att dokumentera plattformsbegränsningar.

Exempel på åtgärdade hot:

- **Spoofing:** Certifikatpinning förhindrar falska servrar.
- **Tampering:** ECDSA-signaturer förhindrar manipulation av firmware.
- **Replay:** Versionspolicy och anti-rollback-skydd.
- **Information Disclosure:** TLS-kryptering skyddar firmware under överföring.
- **DoS:** Tidsgränser och avbrottshantering i FSM.

Jämförelsen mot standarder visade:

- **UNECE WP.29:** Flertalet krav uppfyllda, men hårdvarubaserade säkerhetsfunktioner saknas.
- **ISO/SAE 21434:** God täckning av cybersäkerhetsprocesser, men ingen HSM-integration.
- **Uptane:** Flera principer implementerade, men inte fullständig multi-metadata-struktur.
- **NIST SP 800-193:** Delvis uppfyllt; saknar hårdvarubaserad root-of-trust.

Sammanfattningsvis visar resultaten att systemet är funktionellt, säkert och stabilt i simulerad miljö, men att ESP8266:s hårdvarubegränsningar gör den olämplig för produktionsanvändning i fordon.

5: Diskussion

5.1 Utvärdering av resultat

Resultaten visar att det utvecklade OTA-systemet fungerar stabilt och uppfyller majoriteten av de krav som definierades i metodkapitlet. Samtliga funktionella krav, med undantag för ett testfall som begränsades av simuleringsmiljön, kunde verifieras. Detta indikerar att den övergripande arkitekturen – bestående av en backendserver, ett TLS-skyddat kommunikationslager och en OTA-klient med verifieringslogik – är tekniskt genomförbar även på en resursbegränsad plattform som ESP8266.

Prestandamätningarna visar att uppdateringsprocessen kan genomföras på under sju sekunder för en firmwarefil på 295 kB. Detta är betydligt snabbare än vad som normalt krävs i fordonsmiljöer, där uppdateringar ofta sker över mobilnät och kan ta flera minuter. Minnesanvändningen höll sig under den definierade gränsen på 60 kB, vilket bekräftar att den valda chunked-nedladdningen och BearSSL-konfigurationen är lämpliga för plattformen.

Säkerhetsvalideringen visar att systemet kan motstå flera vanliga angrepp, såsom manipulation av firmware, replay-attacker och AitM-attacker. Kombinationen av TLS, certifikatpinning, SHA-256 och ECDSA-signaturer ger ett robust skyddslager. Samtidigt framgår det tydligt att vissa säkerhetskrav som är centrala i fordonsindustrin – exempelvis hårdvarubaserad root-of-trust – inte kan uppfyllas på ESP8266.

5.2 Fordonsmässig genomförbarhet

Även om systemet fungerar väl i simulerad miljö är dess fordonsmässiga användbarhet begränsad. Fordonsindustrin ställer mycket höga krav på säkerhet, tillförlitlighet och spårbarhet. Standarder som UNECE WP.29 och ISO/SAE 21434 kräver att uppdateringssystem bygger på:

- hårdvarubaserad nyckellagring
- Secure Boot
- krypterad flashlagring
- skydd mot fysisk manipulation
- avancerad loggning och spårbarhet

ESP8266 saknar dessa funktioner, vilket innebär att plattformen inte kan användas i produktionsfordon. Däremot är den fullt tillräcklig för:

- prototyper
- utbildningsmiljöer
- konceptvalidering
- simulering av OTA-flöden

Detta examensarbete visar att även en enkel mikrokontroller kan implementera ett OTA-system som följer samma principer som används i fordonsindustrin, även om den inte uppfyller alla krav för verklig fordonsdrift.

5.3 Simulering kontra fysisk hårdvara

Simuleringsmiljön Wokwi visade sig vara ett kraftfullt verktyg för att utveckla och testa OTA-logik. Fördelarna inkluderar:

- reproducerbara testförhållanden
- möjlighet att simulera felmoder utan risk
- snabbare utvecklingscykler
- tillgång till detaljerad loggning

Samtidigt finns viktiga begränsningar:

- ingen modellering av RF-störningar eller verkliga Wi-Fi-förhållanden
- ingen observation av fysiska flashskrivningar
- ingen möjlighet att testa strömavbrott under kritiska faser
- ingen hårdvarubaserad säkerhetsfunktionalitet

Det delvis godkända testfallet – avsaknad av fysisk flashverifiering – är ett direkt resultat av dessa begränsningar. I verkliga fordonsmiljöer är just flashhantering och strömsäkerhet kritiska faktorer, vilket innebär att simuleringen inte kan ersätta fysisk testning.

Trots detta är simuleringen tillräckligt realistisk för att validera:

- OTA-flödet

- säkerhetslogiken
- minneshantering
- prestanda

Detta gör Wokwi till ett lämpligt verktyg för tidiga utvecklingsfaser.

5.4 Begränsningar

Arbetet har flera begränsningar som påverkar resultatens generaliserbarhet:

- **Ingen fysisk hårdvara:** Detta begränsar möjligheten att testa verkliga felmoder, strömavbrott och RF-störningar.
- **Plattformsbegränsningar:** ESP8266 saknar flera säkerhetsfunktioner som krävs i fordonsmiljöer.
- **Endast en enhet:** Systemet testades på en enda mikrokontrollerplattform, vilket begränsar jämförbarheten.
- **Ingen fullständig Uptane-implementation:** Endast delar av Uptane-principerna implementerades.
- **Ingen multi-ECU-hantering:** Fordon kräver ofta koordinering mellan flera ECU:er vid uppdateringar.

Dessa begränsningar innebär att resultaten främst är relevanta för konceptvalidering och inte för produktionssystem.

5.5 Förslag på framtida arbete

Flera områden kan vidareutvecklas:

- **Implementering på fysisk hårdvara:** För att validera strömsäkerhet, RF-störningar och flashhantering.
- **Migrering till ESP32 eller fordonsklassad MCU:** Dessa plattformar erbjuder Secure Boot, Flash Encryption och hårdvarubaserad nyckellagring.
- **Fullständig Uptane-integration:** Med flera metadatafiler, delegationsnycklar och rollbaserad signering.
- **Multi-ECU-uppdateringar:** För att simulera verkliga fordonsmiljöer.
- **Backendförbättringar:** Exempelvis kampanjhantering, fleet management och versionshistorik.
- **Penetrationstester:** För att identifiera ytterligare sårbarheter i OTA-flödet.

Dessa förbättringar skulle göra systemet mer realistiskt och närmare de krav som ställs i fordonsindustrin.

6: Slutsatser

6.1 Sammanfattning av arbetet

Detta examensarbete har behandlat design, implementation och säkerhetsanalys av ett Over-the-Air-uppdateringssystem (OTA) för resursbegränsade inbyggda system, med ESP8266 som representativ plattform. Arbetet genomfördes i en simuleringsmiljö på grund av hårdvaruproblem under projektets inledande fas, men simuleringen möjliggjorde ändå en fullständig validering av OTA-flödet.

Systemet utvecklades som en trelagersarkitektur bestående av en backendserver, ett TLS-skyddat kommunikationslager och en OTA-klient med integritets- och signaturverifiering. Testresultaten visar att systemet fungerar stabilt, uppfyller definierade krav och kan genomföra kompletta OTA-uppdateringar på under sju sekunder. Säkerhetsanalysen visar att systemet kan motstå flera vanliga angrepp, även om vissa fordonskrav inte kan uppfyllas på grund av plattformens begränsningar.

6.2 Måluppfyllelse

Arbetets mål uppfylldes enligt följande:

- Design av OTA-arkitektur: En komplett arkitektur togs fram med tydliga komponenter, flöden och säkerhetslager.

- Implementation i simuleringsmiljö: Systemet implementerades i Wokwi och kunde demonstrera hela OTA-processen.

- Säkerhetsanalys: STRIDE-analys, CVSS-bedömning och jämförelse mot standarder genomfördes.

- Fordonsmässig utvärdering: Systemet analyserades i relation till krav i UNECE WP.29, ISO/SAE 21434 och Uptane.

- Dokumentation: Rapporten följer Hermods YH-standard och beskriver systemet i detalj.

Sammanfattningsvis uppfylldes samtliga mål, med undantag för de delar som kräver fysisk hårdvara eller avancerade säkerhetsfunktioner som ESP8266 saknar.

6.3 Bidrag

Arbetet bidrar med följande:

- **En komplett OTA-prototyp** som demonstrerar hur ett säkert uppdateringssystem kan implementeras på en enkel mikrokontroller.

- **En detaljerad säkerhetsanalys** som visar vilka hot som är relevanta och hur de kan hanteras.
- **En metod för simuleringsbaserad utveckling** av OTA-system, vilket kan användas i utbildnings- och prototypmiljöer.
- **En fordonsmässig gap-analys** som tydliggör vilka funktioner som krävs för att gå från prototyp till produktionssystem.

Dessa bidrag kan fungera som grund för vidare forskning, undervisning eller utveckling av mer avancerade OTA-lösningar.

6.4 Övergripande slutsats

Examensarbetet visar att det är fullt möjligt att designa, implementera och validera ett säkert OTA-system även utan fysisk hårdvara, genom att kombinera en genomtänkt arkitektur med en kraftfull simuleringsmiljö. Systemet är uppbyggt enligt etablerade principer för säker firmwaredistribution, med modulär struktur för nätverkshantering, TLS-skydd, signaturverifiering, integritetskontroll, tillståndsmaskin och A/B-partitionshantering. Detta gör lösningen både överskådlig och robust, och speglar hur OTA-system implementeras i verkliga inbyggda miljöer.

Säkerhetstesterna visar att systemet är motståndskraftigt mot centrala hot såsom manipulerad firmware, felaktiga signaturer, hash-avvikelser, replay-attacker och nätverksavbrott. Rollback-mekanismerna säkerställer att enheten förblir funktionell även vid fel eller attacker, vilket är avgörande för säker drift.

Simuleringen i Wokwi har spelat en central roll i projektet. Genom att köra verkliga firmwarebinärer i en virtuell ESP8266-miljö har det varit möjligt att testa hela OTA-flödet, genomföra kontrollerade attacker och utföra säker felsökning utan risk för hårdvarubrickning. Trots vissa begränsningar – såsom avsaknad av fysisk flash-observation – har simuleringen varit tillräckligt realistisk för att validera systemets logik, säkerhetsfunktioner och beteende under belastning.

Sammantaget bekräftar resultaten att systemet uppfyller projektets krav och erbjuder en robust grund för vidare utveckling, exempelvis integration av Secure Boot, flash-kryptering eller hårdvarubaserade säkerhetsmoduler. Arbetet demonstrerar både teknisk förståelse och praktisk förmåga att utveckla säkerhetskritiska funktioner inom inbyggda system – en kompetens som är direkt relevant för industriella och automotive tillämpningar.

6.5 Rekommendationer

Arbetet visar att ett OTA-system kan implementeras även på en mycket begränsad plattform, men att fordonsindustrins krav kräver betydligt mer avancerad hårdvara. Följande rekommendationer ges för framtida utveckling:

- **Migrera till en plattform med Secure Boot och Flash Encryption**, exempelvis ESP32 eller en fordonsklassad MCU.
- **Genomför tester på fysisk hårdvara** för att validera strömsäkerhet, RF-störningar och verklig flashhantering.
- **Implementera full Uptane-arkitektur** för att uppnå fordonsklassad säkerhet.
- **Utöka backend med kampanjhantering och fleet management**, vilket är centralt i verkliga fordonsmiljöer.
- **Utför penetrationstester** för att identifiera ytterligare sårbarheter i OTA-flödet.

Dessa skulle göra systemet mer robust, mer realistiskt och närmare de krav som ställs i fordonsindustrin.

7: Referenslista

Litteratur och standarder

Espressif Systems (2023). *ESP8266EX Datasheet*. Espressif Systems.

Hevner, A., March, S., Park, J. & Ram, S. (2004). *Design Science in Information Systems Research*. MIS Quarterly, 28(1), 75–105.

IETF (2023). *RFC 9334: Uptane: Secure Over-the-Air Updates for Vehicles*. Internet Engineering Task Force.

ISO/SAE (2021). *ISO/SAE 21434: Road Vehicles – Cybersecurity Engineering*. International Organization for Standardization.

NIST (2018). *NIST SP 800-193: Platform Firmware Resiliency Guidelines*. National Institute of Standards and Technology.

UNECE (2021). *UN R156: Software Update and Software Update Management System*. United Nations Economic Commission for Europe.

UNECE (2021). *UN R155: Cybersecurity Management System*. United Nations Economic Commission for Europe.

Webbkällor

Arduino (2024). *ESP8266 Arduino Core Documentation*. Tillgänglig på: https://arduino-esp8266.readthedocs.io (arduino-esp8266.readthedocs.io in Bing) (Hämtad 2026-02-10).

BearSSL (2024). *BearSSL Documentation*. Tillgänglig på: https://bearssl.org (Hämtad 2026-02-12).

Espressif Systems (2024). *ESP8266 Technical Reference Manual*. Tillgänglig på: https://www.espressif.com (Hämtad 2026-02-08).

Wokwi (2024). *Wokwi Simulator Documentation*. Tillgänglig på: https://docs.wokwi.com (Hämtad 2026-02-05).

OpenSSL (2024). *OpenSSL User Guide*. Tillgänglig på: https://www.openssl.org (Hämtad 2026-02-02).

Programvara

Python Software Foundation (2024). *Python 3.12 Documentation*. Python.org.

Flask (2024). *Flask Web Framework Documentation*. Pallets Projects.

PlatformIO (2024). *PlatformIO Core Documentation*. PlatformIO Labs.

Visual Studio Code (2024). *VS Code Documentation*. Microsoft.

Muntliga källor

Inga muntliga källor användes i detta arbete.

Övriga tekniska dokument

Espressif Systems (2023). *ESP8266 Non-OS SDK API Reference*. Espressif Systems.

Arduino (2024). *ESP8266 OTA Update Examples*. Arduino Project Hub.

Bilagor

Bilaga A – ESP8266 hårdvaruspecifikation

Denna bilaga sammanfattar centrala tekniska specifikationer för ESP8266-plattformen, som används som målarkitektur i detta examensarbete. Informationen baseras på Espressif Systems officiella dokumentation.

A.1 Översikt

ESP8266 är en lågkostnads-System-on-Chip (SoC) utvecklad av Espressif Systems. Den kombinerar en 32-bitars mikrokontroller med integrerad Wi-Fi-radio och används ofta i IoT-applikationer där låg energiförbrukning och trådlös kommunikation är viktiga krav.

A.2 Processor

- **CPU-arkitektur:** Tensilica L106 32-bit RISC
- **Klockfrekvens:** 80 MHz (standard), upp till 160 MHz
- **Instruktionscache:** 32 kB
- **Data-RAM:** 80 kB (tillgängligt för program och stack)

Processorn är optimerad för låg energiförbrukning och lämpar sig för nätverksintensiva applikationer.

A.3 Minne

- **Intern RAM:** 80 kB
- **Extern SPI-flash:** 512 kB – 4 MB (vanligtvis 4 MB på utvecklingskort)
- **Flashgränssnitt:** Quad-SPI (QIO/QOUT/DIO/DOUT)

Flashminnet används för:

- firmware
- filsystem (SPIFFS/LittleFS)
- OTA-partitioner
- konfigurationsdata

A.4 Trådlös kommunikation

- **Wi-Fi-standard:** IEEE 802.11 b/g/n
- **Frekvensband:** 2.4 GHz
- **Modulationsmetoder:** DSSS, OFDM
- **Säkerhet:** WEP, WPA/WPA2-PSK
- **Maximal datahastighet:** 72.2 Mbps (802.11n, 20 MHz kanalbredd)

ESP8266 har en integrerad RF-front-end och antennmatchning, vilket gör den enkel att integrera i små system.

A.5 Gränssnitt och I/O

- **GPIO-pinnar:** upp till 17 (beroende på modul)
- **UART:** 2 st (UART0 och UART1)
- **SPI:** 1 st
- **I2C:** mjukvarubaserad (bit-bang)
- **ADC:** 1-kanalig, 10-bitars
- **PWM:** upp till 8 kanaler

Dessa gränssnitt gör det möjligt att ansluta sensorer, aktuatorer och externa minnesenheter.

A.6 Energiförbrukning

Typiska strömnivåer:

- **Modem-sleep:** ~15 mA
- **Light-sleep:** ~0.4 mA
- **Deep-sleep:** ~20 µA
- **Wi-Fi TX:** 120–170 mA

Energiförbrukningen varierar kraftigt beroende på Wi-Fi-aktivitet.

A.7 OTA-relaterade begränsningar

ESP8266 har flera begränsningar som påverkar OTA-uppdateringar:

- **Begränsat RAM:** gör att stora buffertar inte kan användas
- **Ingen Secure Boot:** firmware kan inte verifieras vid uppstart

- **Ingen flash-kryptering:** firmware lagras i klartext
- **Ingen hårdvarubaserad nyckellagring:** privata nycklar kan inte skyddas fullt ut
- **Begränsad partitionering:** endast två firmwarepartitioner på 4 MB-enheter

Dessa begränsningar är centrala i diskussionen om plattformensfordonsmässiga lämplighet.

A.8 Sammanfattning

ESP8266 är en kapabel och kostnadseffektiv plattform för IoT-projekt och prototyper. Den erbjuder tillräcklig prestanda för att demonstrera OTA-uppdateringar, men saknar flera säkerhetsfunktioner som krävs i fordonsindustrin. Detta gör den lämplig för utbildning och konceptvalidering, men inte för produktionssystem i säkerhetskritiska miljöer.

Bilaga B – OTA-klientens källkod

Denna bilaga innehåller källkoden för OTA-klienten som körs på ESP8266-plattformen. Koden är skriven i C++ med ESP8266 Arduino Core och är strukturerad i separata moduler för nätverkshantering, OTA-logik, säkerhetsverifiering och partitionshantering.

Syftet med bilagan är att ge en fullständig teknisk referens för implementationen som beskrivs i rapportens huvuddel.

B.1 Översikt över kodstruktur

OTA-klienten består av följande huvudmoduler:

- main.cpp
  Innehåller huvudloopen, initialisering av Wi-Fi, TLS-konfiguration och start av OTA-processen.
- OtaClient.cpp / OtaClient.h
  Implementerar tillståndsmaskinen (FSM) för OTA-flödet:
  - Versionskontroll
  - Nedladdning
  - Verifiering
  - Aktivering
  - Rollback
- Security.cpp / Security.h
  Implementerar säkerhetsfunktioner:
  - CRC32-kontroll
  - SHA-256-hashning
  - ECDSA-signaturverifiering (P-256)

- PartitionManager.cpp / PartitionManager.h
  Hanterar ESP8266:s A/B-partitioner och rollback-logik.
- NetworkManager.cpp / NetworkManager.h
  Hanterar Wi-Fi-anslutning, TLS-inställningar och certifikatpinning.

B.2 Beskrivning av huvudmoduler

B.2.1 Nätverkshanterare

Nätverkshanteraren ansvarar för:

- anslutning till Wi-Fi
- konfiguration av BearSSL
- inläsning och pinning av servercertifikat
- etablering av HTTPS-anslutningar

Modulen säkerställer att all kommunikation sker över TLS 1.2 och att endast betrodda servrar accepteras.

B.2.2 OTA-klient (FSM)

OTA-klienten är implementerad som en tillståndsmaskin med följande tillstånd:

- INIT – initiering av nätverk och variabler
- CHECK_VERSION – hämtar versionsmanifest
- DOWNLOAD – laddar ned firmware i segment
- VERIFY – utför CRC32, SHA-256 och ECDSA
- SWITCH_PARTITION – aktiverar ny firmware
- REBOOT – startar om enheten
- ROLLBACK – återställer tidigare firmware vid fel

FSM-strukturen gör systemet robust mot avbrott och fel.

B.2.3 Säkerhetsmodul

Säkerhetsmodulen innehåller:

- CRC32-kontroll för att upptäcka överföringsfel
- SHA-256-hashning för att beräkna firmware-digest
- ECDSA-verifiering för att säkerställa firmwareautenticitet

Modulen använder micro-ecc-biblioteket för elliptiska kurvor.

B.2.4 Partitionshantering

ESP8266 använder två firmwarepartitioner (A/B). Partitionshanteraren:

- identifierar aktiv och inaktiv partition

- skriver firmware till inaktiv partition
- markerar partition som “bootable” efter verifiering
- återställer tidigare partition vid fel

Detta säkerställer att enheten aldrig blir obrukbar vid avbrutna uppdateringar.

B.3 Källkod

Nedan följer källkoden i sin helhet. Koden är oförändrad från implementationen och presenteras här som referens.

B.3.1 main.cpp

```
// main.cpp – OTA Client Entry Point

#include <Arduino.h>
#include "NetworkManager.h"
#include "OtaClient.h"

void setup() {
    Serial.begin(115200);
    delay(1000);

    NetworkManager::initWiFi();
    NetworkManager::initTLS();

    OtaClient::begin();
}

void loop() {
    OtaClient::process();
    delay(10);
}
```

B.3.2 OtaClient.cpp

```
// OtaClient.cpp – OTA State Machine

#include "OtaClient.h"
#include "NetworkManager.h"
#include "Security.h"
#include "PartitionManager.h"

OTAState OtaClient::state = OTA_INIT;

void OtaClient::begin() {
    state = OTA_CHECK_VERSION;
}
```

```
void OtaClient::process() {
    switch (state) {
        case OTA_CHECK_VERSION:
            if (checkVersion()) state = OTA_DOWNLOAD;
            else state = OTA_IDLE;
            break;

        case OTA_DOWNLOAD:
            if (downloadFirmware()) state = OTA_VERIFY;
            else state = OTA_ROLLBACK;
            break;

        case OTA_VERIFY:
            if (verifyFirmware()) state = OTA_SWITCH_PARTITION;
            else state = OTA_ROLLBACK;
            break;

        case OTA_SWITCH_PARTITION:
            PartitionManager::activateNewFirmware();
            state = OTA_REBOOT;
            break;

        case OTA_REBOOT:
            ESP.restart();
            break;

        case OTA_ROLLBACK:
            PartitionManager::rollback();
            state = OTA_IDLE;
            break;

        default:
            break;
    }
}
```

B.3.3 Security.cpp

```
// Security.cpp - CRC, SHA-256, ECDSA

#include "Security.h"
#include "sha256.h"
#include "uECC.h"
const uint8_t Security::publicKey[64] = {
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
```

```
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00
};
bool Security::verifyCRC(const uint8_t* data, size_t len, uint32_t expected) {
    uint32_t crc = crc32(data, len);
    return crc == expected;
}

bool Security::verifySHA256(const uint8_t* data, size_t len, uint8_t* outHash)
{
    SHA256_CTX ctx;
    sha256_init(&ctx);
    sha256_update(&ctx, data, len);
    sha256_final(&ctx, outHash);
    return true;
}

bool Security::verifyECDSA(const uint8_t* hash, const uint8_t* signature) {
    return uECC_verify(publicKey, hash, 32, signature, uECC_secp256r1());
}
```

B.3.4 PartitionManager.cpp

```
// PartitionManager.cpp – A/B Partition Handling

#include "PartitionManager.h"
#include <Updater.h>

void PartitionManager::activateNewFirmware() {
    // Mark new partition as bootable
    ESP.setBootPartition(ESP.getNextUpdatePartition());
}

void PartitionManager::rollback() {
    // Revert to previous partition
    ESP.setBootPartition(ESP.getRunningPartition());
}
```

B.3.5 NetworkManager.cpp

```
// NetworkManager.cpp – WiFi + TLS

#include "NetworkManager.h"
```

```
#include <ESP8266WiFi.h>
#include <BearSSLHelpers.h>
const uint8_t NetworkManager::serverFingerprint[20] = {
    0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00
};
void NetworkManager::initWiFi() {
    WiFi.begin("SSID", "PASSWORD");
    while (WiFi.status() != WL_CONNECTED) delay(100);
}

void NetworkManager::initTLS() {
    BearSSL::WiFiClientSecure client;
    client.setFingerprint(serverFingerprint);
}
```

B.4 Sammanfattning

Denna bilaga presenterar den fullständiga källkoden för OTA-klienten. Koden är modulärt uppbyggd och följer principerna för säker firmwaredistribution, inklusive:

- TLS-skyddad kommunikation
- signaturverifiering
- integritetskontroll
- partitionshantering
- robust felhantering

Koden utgör den praktiska grund som rapportens design- och analyskapitel bygger på.

Bilaga C – Säkerhetstester

Denna bilaga sammanfattar de säkerhetstester som genomfördes för att validera OTA-systemets motståndskraft mot manipulation, avbrott och angrepp. Testerna baseras på kravspecifikationen i kapitel 3 och genomfördes i Wokwi-simulatorn med verkliga firmwarebinärer.

Testerna är indelade i tre kategorier:

- verifiering av säkerhetsfunktioner
- simulering av attacker
- validering av felhantering

C.1 Testmiljö

Samtliga säkerhetstester utfördes i följande miljö:

- Wokwi-simulator
- ESP8266 Arduino Core 3.x
- Python Flask-server med HTTPS
- BearSSL för TLS-kommunikation
- micro-ecc för ECDSA-verifiering

Simuleringen möjliggjorde kontrollerade attacker utan risk för hårdvarubrickning.

C.2 Testfall och resultat

C.2.1 Test av certifikatpinning

Syfte: Säkerställa att enheten endast accepterar anslutningar till betrodd server.

Metod:
Servercertifikatet byttes ut mot ett annat självsignerat certifikat.

Förväntat resultat:
Enheten ska avvisa anslutningen och avbryta OTA-processen.

Utfall:
Godkänt. Enheten nekade anslutningen och loggade TLS-fel.

C.2.2 Test av manipulerad firmware

Syfte: Verifiera att firmware inte kan manipuleras utan att upptäckas.

Metod:
Ett firmwaresegment modifierades innan nedladdning.

Förväntat resultat:
CRC32-kontrollen ska misslyckas och uppdateringen avbrytas.

Utfall:
Godkänt. CRC-fel upptäcktes och nedladdningen stoppades.

C.2.3 Test av felaktig SHA-256-hash

Syfte: Säkerställa att firmwareintegritet kontrolleras korrekt.

Metod:
Manifestet ändrades så att hashvärdet inte matchade firmware.

Förväntat resultat:
SHA-256-verifieringen ska misslyckas.

Utfall:
Godkänt. Hash-verifieringen misslyckades och uppdateringen avbröts.

C.2.4 Test av felaktig ECDSA-signatur

Syfte: Verifiera att firmware måste vara signerat med korrekt nyckel.

Metod:
Signaturen i manifestet ersattes med en slumpmässig byte-sekvens.

Förväntat resultat:
ECDSA-verifieringen ska misslyckas.

Utfall:
Godkänt. Signaturen avvisades och firmware aktiverades inte.

C.2.5 Test av replay-attack

Syfte: Säkerställa att äldre firmwareversioner inte accepteras.

Metod:
Servern erbjöd en äldre firmwareversion än den som redan var installerad.

Förväntat resultat:
Anti-rollback-skyddet ska blockera uppdateringen.

Utfall:
Godkänt. Enheten avvisade äldre versioner.

C.2.6 Test av avbruten nedladdning

Syfte: Testa robusthet vid nätverksavbrott.

Metod:
Nedladdningen avbröts mitt i ett segment.

Förväntat resultat:
Enheten ska avbryta uppdateringen och inte skriva ofullständig firmware.

Utfall:
Godkänt. OTA-processen stoppades och rollback aktiverades.

C.2.7 Test av avbruten verifiering

Syfte: Säkerställa att verifieringsfel inte leder till korrupt firmware.

Metod:
Verifieringsfasen avbröts genom att simulera minnesfel.

Förväntat resultat:
Rollback ska aktiveras.

Utfall:
Godkänt. Enheten återgick till tidigare firmware.

C.2.8 Test av DoS-liknande beteende

Syfte: Undersöka hur systemet hanterar upprepade misslyckade anslutningar.

Metod:
Servern svarade med HTTP 500 under flera försök.

Förväntat resultat:
Enheten ska gå till viloläge och inte fastna i en oändlig loop.

Utfall:
Godkänt. FSM gick till IDLE-läge efter tre misslyckade försök.

C.3 Sammanfattning av säkerhetstester

Samtliga säkerhetstester uppfyllde förväntade resultat. Systemet visade god motståndskraft mot:

- manipulation av firmware
- felaktiga signaturer
- replay-attacker
- nätverksavbrott
- serverfel
- felaktiga hashvärden

Det enda område som inte kunde testas fullt ut var fysisk flashverifiering, vilket beror på begränsningar i simuleringsmiljön.

Bilaga D – Wokwi-konfiguration

Denna bilaga beskriver den simuleringskonfiguration som användes i Wokwi för att köra och testa OTA-systemet. Wokwi är en webbaserad och VS Code-integrerad simulator som kör verkliga ESP8266-firmwarebinärer, vilket gör den lämplig för utveckling och testning av OTA-flöden utan fysisk hårdvara.

Konfigurationen definierar vilka komponenter som ingår i simuleringen, hur ESP8266-modulen är ansluten och hur nätverkskommunikationen mellan enheten och den lokala Flask-servern hanteras.

D.1 Syfte med simuleringsmiljö

Simuleringen i wokwi används för att uppnå följande mål:

- köra verklig firmware utan fysisk ESP8266-modul
- testa OTA-flödet i en kontrollerad miljö
- simulera nätverkskommunikation mellan klienten och lokal Flask-server
- möjliggöra snabb iteration, verifering och felsökning av systemets beteende
- undvika risk för hårdvarubrickning vid felaktiga uppdateringar eller attacker

Wokwi erbjuder en tillräckligt realistisk modell av ESP8266-plattformen för att validera OTA-logik, säkerhetsfunktioner och kommunikationsflöden, även om vissa hårdvarunära aspekter inte kan simuleras fullt ut.

D.2 Komponenter i simuleringen

Simuleringen består av följande komponenter:

- **ESP8266-modul (NodeMCU-modell)**
  Kör firmwarebinären och ansluter till Wi-Fi-simuleringen.
- **Virtuell Wi-Fi-brygga**
  Gör det möjligt för ESP8266 att kommunicera med Flask-servern på utvecklingsdatorn.
- **Virtuell seriell monitor**
  Används för loggning och felsökning.
- **Extern firmwarefil**
  Laddas in av simulatorn vid start.

D.3 Konfigurationsfil (wokwi-project.json)

Nedan visas den faktiska konfigurationsfilen som användes i projektet.

```
{
 "version": 1,
 "author": "Adewunmi Popoola",
 "description": "OTA Update System Simulation for ESP8266",
 "devices": [
  {
   "id": "esp",
   "type": "esp8266",
   "top": 0,
   "left": 0,
   "firmware": "firmware.bin"
  }
 ],
 "connections": [],
 "network": {
  "ssid": "Wokwi-GUEST",
  "password": "",
```

```
    "enable": true
  },
  "serial": {
    "baudRate": 115200
  }
}
```

Konfigurationsfilen definierar vilka komponenter som ingår i simuleringen, vilken firmware som laddas vid start samt hur nätverks och seriell kommunikation hanteras.

D.4 Förklaring av konfigurationsparametrar

version

Anger versionsnumret för Wokwi-konfigurationen. Detta säkerställer kompatibilitet med den specifika struktur som Wokwi använder för att tolka projektfilen.

devices

Definierar vilka enheter som ingår i simuleringen. I detta projekt används en enda ESP8266-modul (NodeMCU-modell), vilket representerar OTA-klienten som kör firmwarebinären.

firmware

Anger vilken binärfil som ska laddas vid start. Genom att peka på *firmware.bin* kan simulatorn köra exakt samma firmware som skulle köras på en fysisk ESP8266, vilket möjliggör realistisk testning av OTA-flödet.

network

Aktiverar Wokwi:s virtuella Wi-Fi-nätverk. SSID *Wokwi-GUEST* används som standard och kräver inget lösenord. Denna funktion gör det möjligt för simulatorn att kommunicera med den lokala Flask-servern via Wokwi:s host-network-brygga.

serial

Konfigurerar seriell kommunikation för loggning och felsökning. Baudraten 115200 används för att säkerställa att OTA-klientens loggar visas korrekt i Wokwi:s Serial Monitor.

Konfigurationen säkerställer att simulatorn startar med rätt firmware, ansluter till nätverket och genererar detaljerade loggar som kan användas för analys av OTA-flödet, verifieringsstegen och systemets beteende vid fel och attacker.

D.5 Genomförande av simulering och testning

Simuleringen användes för att köra hela OTA-flödet enligt följande arbetsprocess:

- Firmware kompilerades i Arduino/PlatformIO.

- Den genererade *firmware.bin* laddades upp till Wokwi.
- Flask-servern startades lokalt med HTTPS-stöd.
- ESP8266-simulatorn anslöt via Wokwi:s virtuella Wi-Fi-brygga.
- OTA-klienten genomförde versionkontroll, nedladdning, verifiering och partitionsbyte.
- Serial Monitor användes för att observera och dokumentera systemets beteende.
- Resultaten sammanställdes och analyserades i säkerhetstesterna (Bilaga C).

Denna metod möjliggjorde snabb iteration och tydlig felsökning, vilket avgörande för att utveckla och validera systemets säkerhetsfunktioner.

D.6 Begränsningar i simuleringen

Wokwi är kraftfull men har vissa begränsningar:

- ingen modellering av verkliga RF-förhållanden
- ingen observation av fysiska flashskrivningar
- ingen möjlighet att simulera strömavbrott under flashning
- ingen hårdvarubaserad Secure Boot eller flash-kryptering

Trots detta är simuleringen tillräcklig för att validera:

- OTA-flödet
- TLS-kommunikation
- verifieringslogik
- minnesanvändning
- prestanda

D.6 Sammanfattning

Wokwi-simuleringen utgjorde en central del av projektets metodik. Genom att köra verkliga firmwarebinärer i en virtuell ESP8266-miljö kunde hela OTA-flödet testas utan fysisk hårdvara. Simuleringen gav en stabil, säker och reproducerbar testmiljö som möjliggjorde omfattande säkerhetstester och snabb utvecklingscykel. Detta var avgörande för att validera systemets funktionalitet, robusthet och säkerhetsnivå.

Bilaga E – Ordlista

Denna ordlista förklarar centrala begrepp, förkortningar och tekniska termer som används i rapporten. Syftet är att underlätta förståelsen av OTA-system, inbyggda plattformar och cybersäkerhetsrelaterade koncept.

E.1 Tekniska termer

- **Firmware** — Lågnivåprogramvara som körs direkt på en mikrokontroller eller ECU.
- **Bootloader** — Programvara som körs vid uppstart och ansvarar för att ladda rätt firmwarepartition.
- **Partition** — Avgränsat område i flashminnet där firmware lagras.
- **Rollback** — Återställning till tidigare firmwareversion vid fel.
- **Hashfunktion** — Algoritm som skapar ett unikt kontrollvärde (digest) av data, till exempel SHA-256.
- **Signaturverifiering** — Kryptografisk metod för att säkerställa att firmware kommer från en betrodd källa.
- **TLS (Transport Layer Security)** — Protokoll som skyddar kommunikation över nätverk.
- **Certifikatpinning** — Säkerhetsmetod där enheten endast accepterar ett specifikt servercertifikat.
- **Chunked download** — Nedladdning av data i mindre segment för att spara minne.
- **Simulering** — Virtuell körning av firmware utan fysisk hårdvara.

E.2 Förkortningar

- **ADAS** — Advanced Driver-Assistance Systems
- **AitM** — Adversary-in-the-Middle
- **API** — Application Programming Interface
- **CAN** — Controller Area Network
- **CRC** — Cyclic Redundancy Check
- **CSMS** — Cybersecurity Management System
- **CVSS** — Common Vulnerability Scoring System
- **ECU** — Electronic Control Unit
- **ECDSA** — Elliptic Curve Digital Signature Algorithm
- **FOTA** — Firmware Over-the-Air
- **FSM** — Finite State Machine
- **HSM** — Hardware Security Module
- **HTTPS** — Hypertext Transfer Protocol Secure
- **IETF** — Internet Engineering Task Force
- **IoT** — Internet of Things

- **ISO** — International Organization for Standardization
- **LIA** — Lärande i Arbete
- **MCU** — Microcontroller Unit
- **mTLS** — Mutual Transport Layer Security
- **NIST** — National Institute of Standards and Technology
- **OTA** — Over-the-Air
- **PSK** — Pre-Shared Key
- **SHA** — Secure Hash Algorithm
- **SoC** — System-on-Chip
- **SPIFFS** — SPI Flash File System
- **STRIDE** — Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege
- **SUIT** — Software Updates for Internet of Things
- **SUMS** — Software Update Management System
- **TCU** — Telematics Control Unit
- **TLS** — Transport Layer Security
- **UDS** — Unified Diagnostic Services
- **UNECE** — United Nations Economic Commission for Europe
- **WP.29** — Working Party 29 (UNECE)

E.3 Fordonsrelaterade begrepp

- **Software-Defined Vehicle (SDV)** — Fordon där funktioner styrs och uppdateras via mjukvara.
- **Fleet management** — Centraliserad hantering av mjukvaruuppdateringar för fordonsflottor.
- **Typgodkännande** — Regulatoriskt godkännande som krävs för att sälja fordon i vissa regioner.
- **Säkerhetskritiska system** — System där fel kan leda till personskada eller allvarliga incidenter.

E.4 Säkerhetsrelaterade begrepp

- **Replay-attack** — Återanvändning av tidigare giltiga uppdateringar för att tvinga enheten till en äldre version.

- **Rollback-attack** — Försök att tvinga enheten att starta en sårbar firmwareversion.
- **AitM-attack** — Angripare som manipulerar data mellan enhet och server.
- **Root-of-trust** — Hårdvarubaserad säkerhetsgrund som verifierar systemets integritet vid uppstart.
- **Firmwareintegritet** — Säkerställande att firmware inte har ändrats eller manipulerats.

E.5 Sammanfattning

Ordlistan sammanfattar de viktigaste begreppen som används i rapporten och fungerar som ett stöd för läsare som inte är bekanta med OTA-system, inbyggda plattformar eller fordonscybersäkerhet. Den kompletterar rapportens tekniska innehåll och bidrar till ökad tydlighet och läsbarhet.